

High Level Design (HLD)

Playwright BDD Cucumber Framework · Version 1.1 · TypeScript

1. Purpose

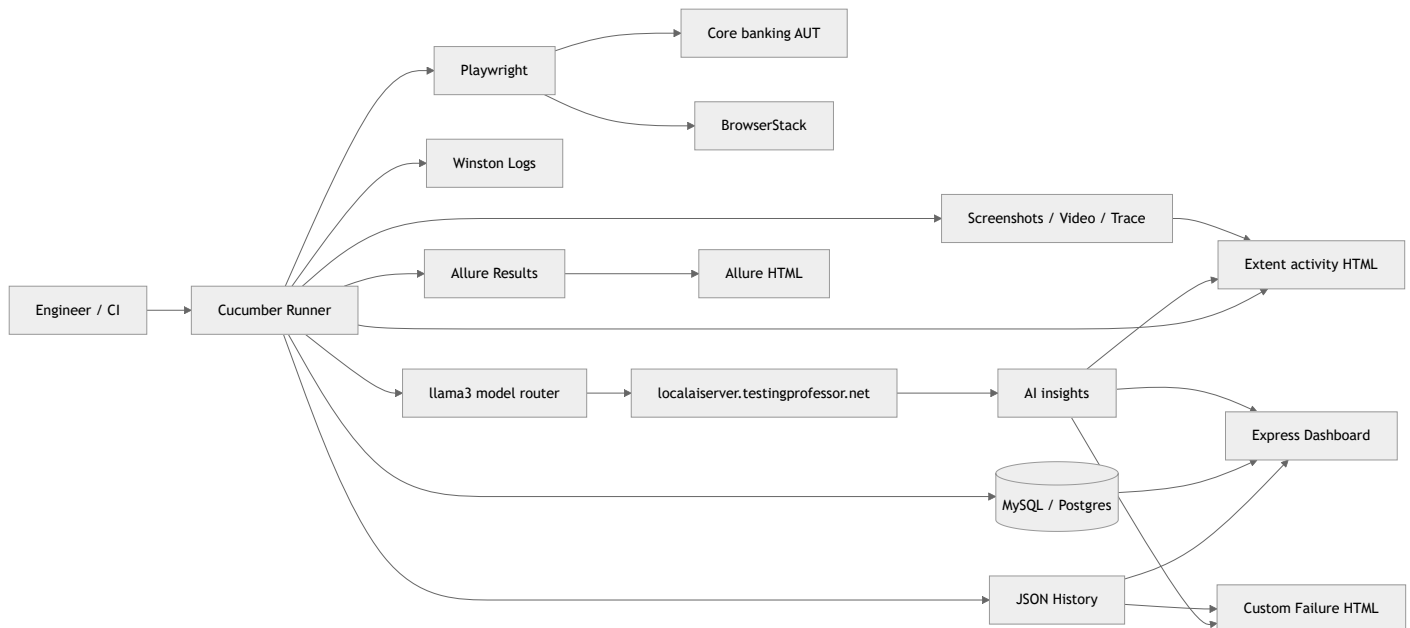
This framework is a reusable UI automation platform using Playwright and Cucumber BDD. It standardizes page objects, actions, logging, error classification, env-driven artifacts (step screenshots, video, traces), Extent-style activity reporting, CI/CD, BrowserStack, database-backed history, and local LLM failure triage with complexity-based model routing.

The current application under test is the core banking site at <https://corebanking.testingprofessor.net/>.

2. Goals

- BDD-first authoring with Gherkin features and step definitions.
- Page Object Model with a single action utility wrapping Playwright APIs.
- Deterministic error taxonomy wrapping Playwright exceptions.
- Worker-safe shared data for parallel and cross-browser runs.
- Allure, custom failure HTML, Extent activity report, and historical dashboard.
- Env-driven per-step screenshots, scenario screenshots, video, and Playwright traces.
- Optional MySQL or PostgreSQL persistence.
- GitHub Actions, Jenkins, Docker, and BrowserStack integration.
- Hosted Ollama LLM triage: llama3 routes work; complex analysis prefers ~32B models, then `gemma4:latest`.

3. System context

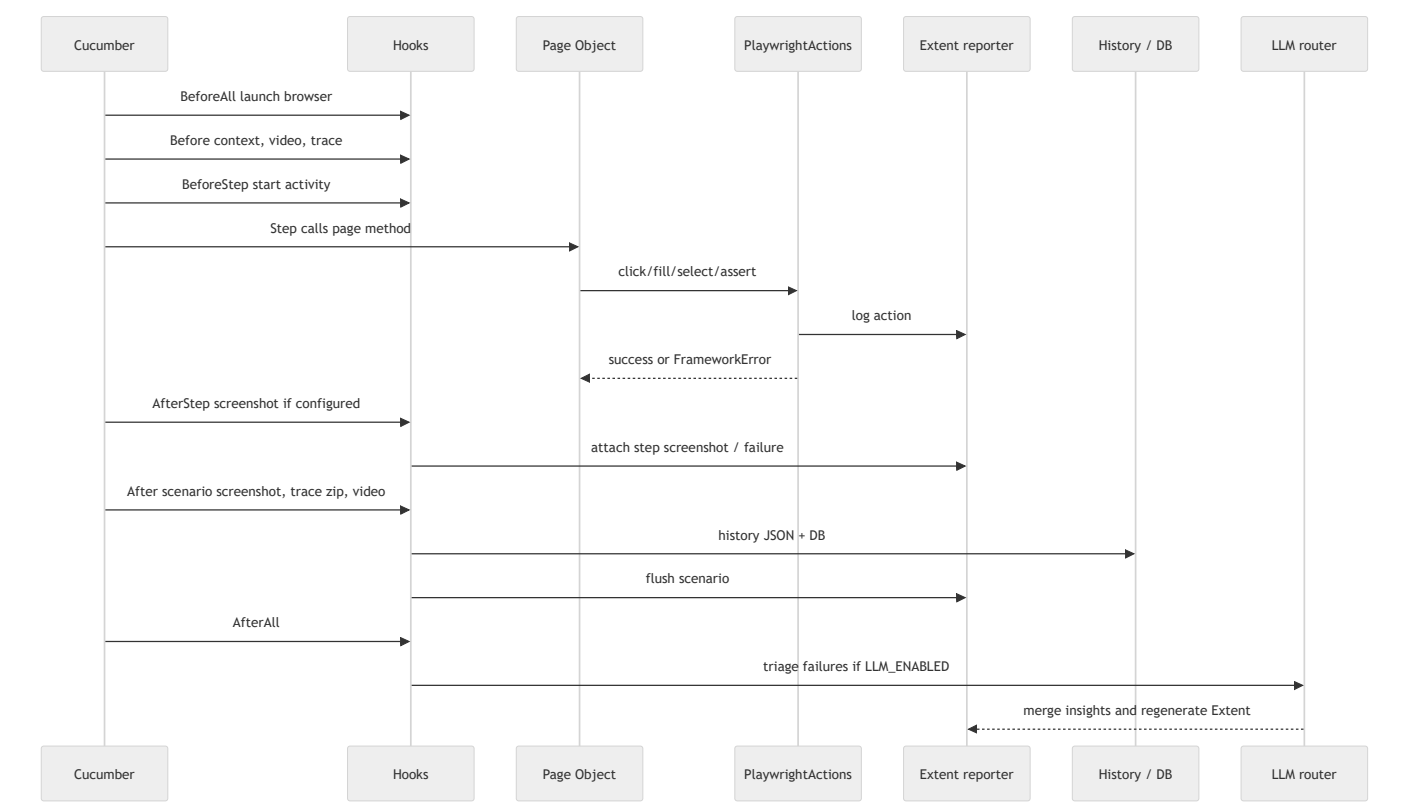


4. Logical architecture

Layer	Components	Responsibility
Authoring	features/*.feature, features/steps	Business-readable scenarios and glue code
Runtime	CustomWorld, hooks, Playwright browser/context/page	Lifecycle, artifacts, correlation IDs
POM	BasePage, LoginPage, AddCustomerPage, CustomersPage, OpenAccountPage, CustomerDashboardPage	Locators and page behaviors for core banking
Actions	PlaywrightActions	Click, fill, select (incl. random), wait, screenshot, mapped errors
Artifacts	hooks + SCREENSHOT / STEP_SCREENSHOT / VIDEO / TRACE	Capture, retain-on-failure, attach to reports
Cross-cutting	Logger, SharedBuffer, FrameworkError	Observability, data sharing, taxonomy
Persistence	DB adapters, history JSON	Run/scenario/failure/insight storage

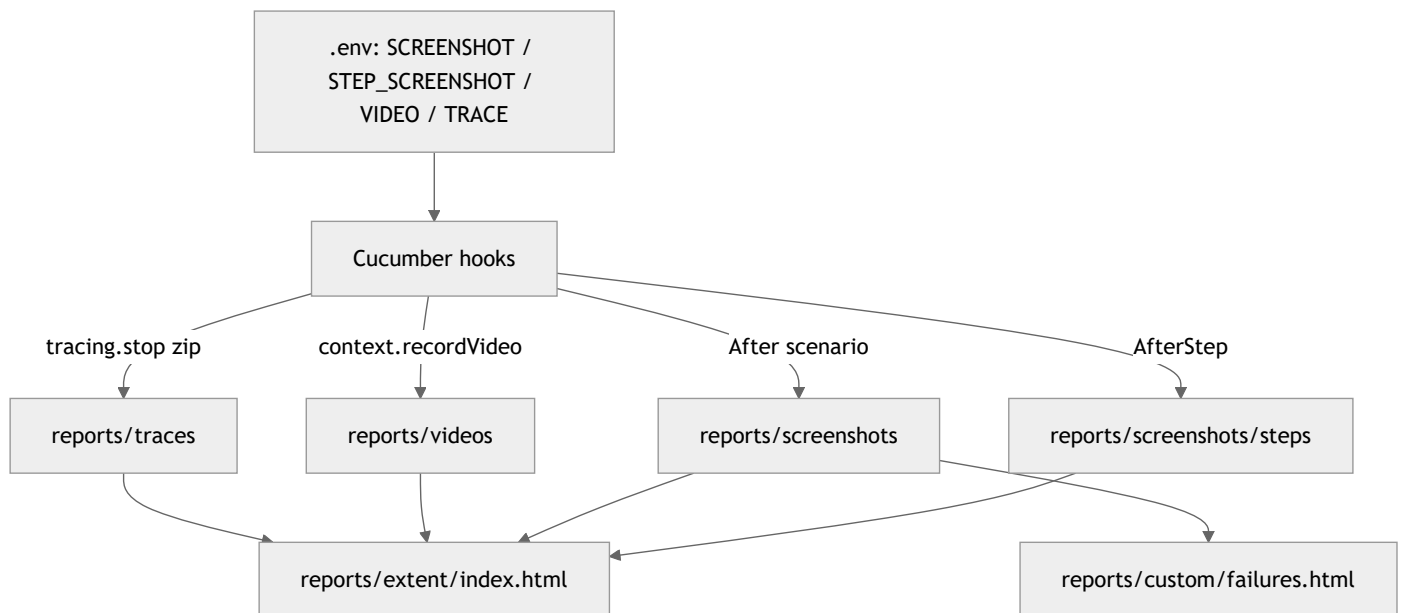
Insights	llmClient, modelRouter, FailureTriage	Catalog models, route by complexity, classify failures
Presentation	Allure, custom HTML, Extent HTML, dashboard	Step activity, screenshots, video, traces, AI notes
Delivery	GitHub Actions, Jenkins, Docker	Repeatable CI/CD

5. Execution flow



6. Artifact capture

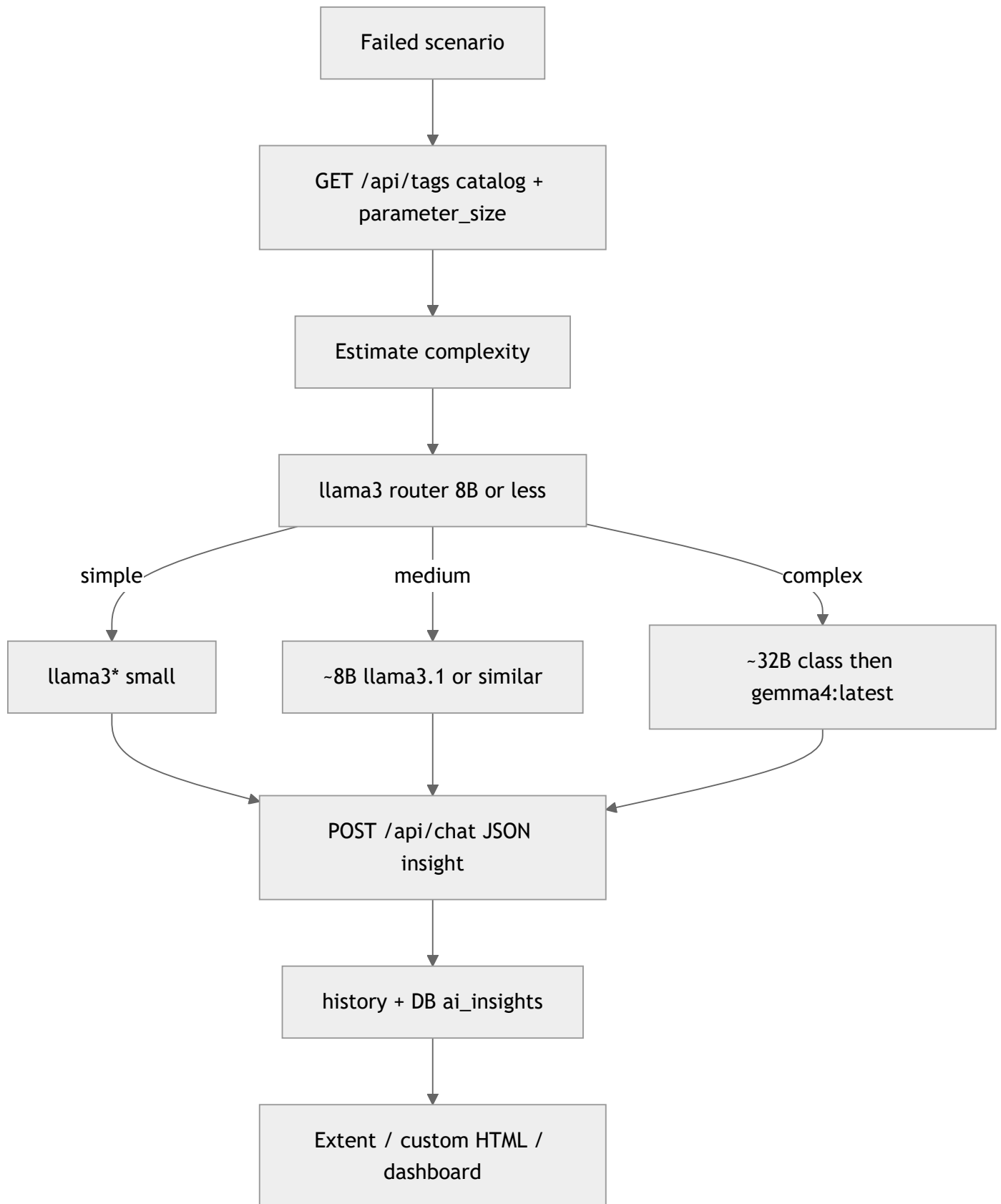
All artifact modes are `off`, `on`, or `retain-on-failure`, driven from `.env`.



Flag	Scope	Output
STEP_SCREENSHOT	After every Gherkin step	reports/screenshots/steps/, inline in Extent
SCREENSHOT	End of scenario	reports/screenshots/
VIDEO	Browser context recording	reports/videos/*.webm
TRACE	Playwright trace	reports/traces/*.zip

7. LLM triage and model routing

The hosted Ollama server is `http://localaiserver.testingprofessor.net`. Calls use `LLM_API_KEY`. Catalog comes from `GET /api/tags` (including `details.parameter_size`).



Complexity	Preferred model
Simple (short timeout / locator miss)	Small llama3* (8B or less), never a 32B model

Medium	~8B class unless the failure looks ambiguous
Complex (long stack, PW_UNKNOWN)	~32B installed model (27B–40B band), else <code>gemma4:latest</code> , else larger

`LLM_MODEL=auto` and `LLM_ROUTE=true` enable routing. `LLM_PREFER_BILLION=32` sets the complex-size target. `LLM_COMPLEX_MODEL=gemma4:latest` is the named fallback.

8. Deployment views

8.1 Local

Node 18+, Playwright browsers, optional MySQL/Postgres. Dashboard on port 3000. LLM defaults to the hosted server `localaiserver.testingprofessor.net` (not a local Ollama process).

8.2 Docker Compose

Services: `mysql`, `postgres`, `ollama` (optional local), `dashboard`, `tests` (profile). Tests write reports to a bind-mounted volume. Hosted LLM can replace compose Ollama via `LLM_HOST`.

8.3 Cloud

When `EXECUTION_ENV=browserstack`, the runner connects via Playwright CDP WebSocket using BrowserStack capabilities instead of launching a local browser.

9. Data sharing

Cucumber workers do not share Node memory. `SharedBuffer` exposes three scopes:

- **scenario** — in-memory Map on World (isolated).
- **worker** — in-memory Map per worker process.
- **global** — locked JSON file plus optional `shared_kv` table so Chrome/Firefox/WebKit and CI shards can exchange keys.

Customer flows persist `customer`, `customers`, `accounts` in scenario and global scopes so later steps (open account, customer login) can reuse generated data.

10. Reporting strategy

Channel	When	Audience
---------	------	----------

Winston files/console	Every action/step	Debugging
Allure	Each run	QA / CI artifacts
Extent activity HTML	End of run (and after triage)	Step + Playwright action log, screenshots, video, trace, AI note
Custom failures.html	Post-run / after triage	Root-cause review
Dashboard	Historical	Leads / trends / flaky tests / AI insights
AI insights	AfterAll when enabled, or dashboard / <code>npm run ai:trriage</code>	Triage acceleration

11. Non-functional requirements

- Isolation: new browser context per scenario.
- Parallel: Cucumber `--parallel` with file-locked global store; AfterAll triage runs on the primary worker only.
- Resilience: DB and LLM are optional; JSON history and Extent still generate.
- Security: credentials and `LLM_API_KEY` only via environment / CI secrets, never committed.
- Extensibility: add a page class + feature + steps without changing core.

12. Constraints and assumptions

- TypeScript on Node 18 or later.
- Cucumber is the test runner; Playwright Test is not used as the runner.
- Core banking pages are implemented; `templates/ExamplePage` remains a scaffold for new pages.
- The local AI server requires a valid `LLM_API_KEY`. HTTPS to that host is not used; HTTP is.
- 32B-class models are detected from Ollama catalog metadata or names such as `:32b`.

13. Related documents

See [Low Level Design \(LLD\)](#) for module contracts, hook behavior, artifact flags, error catalog, database schema, LLM routing, and extension procedures.